1.BERT的全称是 `Bidirectional Encoder Representations from Transformers` ，即基于Transformer的双向编码器表示，BERT在Transformer的基础上做减法，舍弃掉了Decoder，通过堆叠 Encoder，深化模型对文本内容的理解能力，并可以通过添加任务头并进行少量微调的方式得到针对垂直领域的特化模型。主要用来解决自然语言理解相关的问题。

2.BERT的输入是形状为 $B \times T \times C$ 的张量，每个token所对应的向量由token分别经过token embedding、position embedding和segment embedding（用于标记这个token处于哪一句话，不同的句子中的token加上不同的对应向量）后相加得到，经过多层的Encoder后输出形状为 $B \times T \times T$ 的张量再通过线性变换得到 $B \times T \times C$ 的序列输出，每个token的向量包含了与这个token相关的上下文语义信息。如果想要得到某一整句的语义信息，还可以找到句首token所对应的向量并接入一个全连接层和 `tanh` 激活函数，得到一个形状为 $B \times C$ 的总结性张量，作为整个输入序列的池化输出。

3.NSP和MLM分别代表BERT训练的两种模式。NSP（Next Sentence Prediction），即下一句预测，通过给定模型两个句子A和B并判断B是否为A的下一句，来训练模型对整体句意的理解能力和跨句子之间的逻辑性和连贯性；MLM（Masked Language Model），掩码语言模型，俗称完形填空，一般在一个句子中挑选15%的token进行处理遮掩（可能用MASK遮住，也可能替换为随机的token，也可能保持原样），并让模型猜测这些遮掩的token是什么，从而训练模型的语言语义关联。

4.在进行MLM时，如果不进行MASK，模型在进行自注意力时就可以获取到自身的信息，从而在猜词的适合“作弊”，因此需要把需要才的词遮住防止信息泄露。同时，在进行遮掩时不能全用MASK，也会适当使用随机token或者保留原本的token，这是因为真实任务中输入的序列里不会有人为添加的MASK，如果全不使用MASK遮住，模型在猜词时会过度依赖于MASK标记，可能在完成具体任务时表现不佳，而加入适量的随机token和保留原本的token，可以训练模型的纠错能力、并让模型不会变的太过偏向于改掉原来的token。但如果使用过多的随机token甚至全部使用随机token，模型可能会对语义理解产生偏差导致训练结果下降。

5.BERT的预训练先通过大量的数据让模型学会通用的语义理解，再在后训练时用少量数据进行微调适配下游任务。而现实生活中下游任务的数据量是很稀少的，因此直接用于训练模型会产生严重的过拟合，使模型失去泛化能力。

6.BERT的预训练是自监督的，通过自己在文本上挖空或者挑选句子对模型进行训练，且分别进行NSP和MLM这两种任务，最终的损失函数就是这两种任务的交叉熵损失直接相加，最终的训练目标是让模型通过大量的数据学习到语义并可以“理解”文本。通过NSP和MLM分别训练词语语义和句子之间的逻辑等，并将两者结合得到从token到整个输入文段的理解。

7.BERT预训练完成后，需要给BERT接上一个任务头，并在较小学习率下用下游任务的数据进行微调即可。BERT一般用来做自然语言理解类的下游任务，如句子的分类、匹配等任务等。

8.从架构上来看，BERT和GPT虽然都是Transformer架构，但BERT是Encoder-only，训练时做的是双向自注意力，每个token与整个输入序列的每个token建立联系，训练方式是做NSP和MLM这样的判断任务和填词任务；而GPT则是Decoder-only的架构，训练时只能看到这个token以及以前的token来做下一个token的预测任务。

9.BERT的Encoder-only架构不适合用于生成文章，因为BERT在训练过程中看到的是全局所有的token，与预测下一个token这样只能看到前面token的任务不适配。